WHEN CORRECT CODE IS NOT ENOUGH: ASSESSING THE PRODUCTION READINESS OF LLM-GENERATED SOFTWARE

Ramesh Venkatachalapathy

AT&T, McKinney, Texas, USA

ABSTRACT

Large language models (LLMs) increasingly generate executable software, but functional correctness alone does not establish production readiness. This study evaluates GPT-5.6 Sol, Terra, and Luna across 15 algorithmic and production-oriented tasks using five generations per model-task pair, yielding 225 generated programs plus 15 fixed references. The layered testbed combines nominal and hidden-edge-case checks, moderate and severe production-like stress, metamorphic validation, reference-normalized runtime and memory, structural quality, static analysis, implementation-strategy inspection, and generation cost. All 800 functional and 368 applicable robustness checks passed, while production-stress success fell to 95.16% for Sol, 95.60% for Terra, and 93.38% for Luna. Cross-model stress differences were not significant, but generation cost differed strongly. A Quick Sort case showed more than a 500x runtime spread among behaviorally correct implementations. The results show that production-oriented evaluation must extend beyond pass/fail correctness to include stress behavior, resources, implementation strategy, and cost.

KEYWORDS

Large language models, code generation, production readiness, software testing, software quality

1. INTRODUCTION

Large language models are now used in everyday software work. Developers ask them to write functions, create tests, repair defects, explain existing code, and assist with code review. Recent studies cover code generation, automated testing, repair, review, and secure coding [1]–[5], [8]. Once generated code is allowed to enter an application or build pipeline, the evaluation problem changes. The output is no longer only text; it is executable software that may be maintained, integrated, and run under changing workloads.The first check is usually correctness. If a program does not produce the expected result, there is little value in discussing its production use. Passing the available tests, however, does not tell the whole story. Two implementations can return the same outputs and still differ in runtime, memory use, algorithmic strategy, complexity, maintainability, static-analysis findings, and generation cost. These differences become relevant when the code is reused, scaled, audited, or modified.

The problem studied here is what remains to be checked after generated code appears correct. A production-oriented review should expose edge cases, stronger workloads, resource behavior, and implementation choices before the code is treated as ready. This matters especially when nominal tests become easy enough that several model configurations all reach the same pass rate. At that point, correctness by itself has little value for distinguishing the generated implementations.Prior work already covers many pieces of this problem. Researchers have examined mutation testing [3], API misuse [6], performance-oriented generation [7], secure code generation [8], and

DOI: 10.5121/ijsea.2026.17601

maintainability repair [9]. Code-review studies provide another view of generated-software quality [5], [16], [17]. Most of these studies focus on one or a small number of quality dimensions. In practice, a reviewer needs to see several kinds of evidence together: behavior, robustness, runtime, memory, maintainability, security indicators, implementation strategy, and cost.

This study evaluates those dimensions in one controlled testbed. The same frozen program is first checked for nominal behavior and hidden edge cases, then exposed to moderate and severe production-like stress. Where a meaningful relation exists, metamorphic checks are also applied. Runtime and memory are compared with fixed task references, structural measures and static-analysis findings are recorded, implementation strategy is inspected, and generation cost is retained. The workload mixes canonical algorithms with production-oriented tasks so that results from familiar coding problems can be compared with parsing, state, resilience, validation, and batching behavior. Generated source code is frozen before the stronger tests are applied, and generated and reference implementations run in the same environment.

The study makes five contributions. It treats nominal correctness as an entry condition rather than the final score. It evaluates six algorithmic and nine production-oriented tasks under the same repeated protocol. It adds moderate and severe stress, task-specific metamorphic checks, and a Production-Readiness Gap (PRG) that measures the loss from nominal success to production-stress success. It uses fixed references to normalize runtime, memory, and maintainability observations across 225 generated programs. Finally, it records implementation-strategy divergence because two programs can behave the same while using very different computational approaches.Section 2 reviews related work. Section 3 describes the testbed, workload, measurements, and statistical procedure. Section 4 reports the experimental results. Section 5 discusses what the results mean and the limits of the study. Section 6 concludes the paper.

2. RELATED WORK

Research on LLM-generated code now spans testing, repair, review, security, performance, and broader software quality. Holistic evaluations caution against reducing code quality to a single benchmark score [10], while controlled studies show that outcomes vary with task type, prompt design, and model configuration [11], [12]. These findings support evaluation designs that preserve executable evidence rather than relying on a single aggregate score. Execution-based validation is central because generated code must work rather than merely appear plausible. Prior research covers LLM-based unit-test generation and broader testing practice [1], [2], mutation testing [3], systematic task variants [13], iterative feedback and validation loops [14], and bug finding across multiple error classes [15]. Other studies examine maintainability repair [9], secure code generation [8], API misuse [6], and code-review support [5], [16], [17]. Together, these works establish useful quality checks, but they usually evaluate the dimensions separately.

Non-functional evidence becomes important once generated code is expected to run repeatedly or at scale. Performance-aligned generation has been studied for high-performance computing [7], and a large empirical comparison examined defects, vulnerabilities, and complexity in human- and AI-generated code [19]. Function-calling benchmarks emphasize executable interaction rather than text quality alone [20], while agent-security research shows that tool-enabled systems introduce operational risks that must be evaluated at runtime [18]. The gap addressed in this study appears after nominal correctness stops separating the models. Existing work provides strong evidence for testing, security, maintainability, performance, and broader code quality, but these dimensions are usually evaluated in isolation. This study therefore combines a mixed workload, task-specific moderate and severe stress, metamorphic checks, fixed-reference normalization, structural and static-analysis evidence, a Production-Readiness Gap (PRG),

implementation-strategy classification, and generation cost on the same frozen generated artifacts. Table 1 positions this layered evaluation against representative prior studies; the objective is not another isolated score, but a practical view of what evidence remains useful after the initial pass/fail result has saturated.

Recent IJSEA studies reinforce the need to evaluate software beyond headline correctness. Samal et al. [21] frame autonomous software-agent assessment around deployment readiness, repeated runs, robustness, and operational efficiency. Yadavali [22] emphasizes workload variation, resource behavior, and performance assurance, while Khankhoje [23] discusses AI-assisted API testing for diverse inputs, anomaly detection, and software reliability. These studies complement the present focus on stress behavior, resource evidence, and implementation strategy after nominal correctness has saturated.

Table 1: Positioning of Representative LLM-Code Evaluation Studies

Study	**Primary emphasis**	**Evaluation evidence**	**Distinction from the present study**
Schaefer et al. [2]	Automated unit-test generation	Coverage, generated-test validity, and failed-test repair	Evaluates test generation rather than layered readiness of generated application code
Tip et al. [3]	LLM-guided mutation testing	Mutant realism, runtime, cost, and test-suite adequacy	Evaluates mutation-testing effectiveness rather than post-correctness production readiness
Zhao et al. [8]	Secure code generation	CWE-guided vulnerability mitigation across languages and LLMs	Security-specific evaluation rather than multi-layer stress, resource, and strategy evidence
Nichols et al. [7]	Performance-aligned code generation	Serial/OpenMP benchmark speedup and performance alignment	Performance-focused evaluation rather than severity-layered readiness assessment
Holl and Kufer [10]	Holistic code-generation quality	Correctness, runtime efficiency, complexity, and maintainability	Broad quality view, but without task-specific stress tiers, PRG, and explicit strategy classification
This study	Layered production readiness	Nominal + stress + metamorphic + reference-normalized + structural + cost evidence	Integrates evidence after nominal correctness saturates and quantifies hidden readiness loss

3. PRODUCTION-LIKE TESTBED AND EVALUATION METHODOLOGY

3.1. Study Design and Research Questions

The testbed is intended to examine production-oriented evidence, not to create a model leaderboard. Nominal correctness is checked first. The same frozen program is then evaluated for hidden edge cases, moderate and severe stress, metamorphic consistency where applicable, resource behavior against a fixed reference, structural quality, static-analysis findings,

implementation strategy, and cost. This separation makes it possible to ask two different questions: does the program work under the nominal tests, and what changes when the test conditions become stronger? Fixed references are used so that non-functional observations are not based only on model-to-model comparisons.

RQ1: To what extent does nominal functional correctness saturate across the evaluated models and workload classes?

RQ2: How much additional failure evidence is revealed by moderate and severe production-like stress, and what Production-Readiness Gap remains after nominal success?

RQ3: Do algorithmic and production-oriented workloads exhibit different resilience and non-functional quality profiles?

RQ4: How do generated implementations compare with fixed references in runtime, memory, maintainability, and task-appropriate metamorphic behavior?

RQ5: Can behaviorally equivalent generations diverge in implementation strategy and runtime behavior?

RQ6: What quality-cost trade-offs remain when nominal behavioral success is held constant?

3.2. Workload Design

The workload contains 15 tasks: six algorithmic tasks and nine production-oriented tasks. The algorithmic set provides controlled behavior and familiar performance baselines. The production-oriented set covers parsing, state handling, retry behavior, traffic control, input validation, and batching. Table 2 groups the tasks into six families to keep the presentation compact while preserving the distinction between the two workload classes.

Table 2: Experimental Workload and Production-Readiness Dimensions

Workload family	**Class**	**Tasks included**	**No.**	**Evaluation focus**
Search & selection	Algorithmic	Binary Search; Top-K	2	Correctness, boundaries, efficiency
Sorting & interval processing	Algorithmic	Merge Sort; Quick Sort; Heap Sort; Interval Merge	4	Correctness, runtime, memory, strategy
Data transformation	Production-oriented	JSON Normalization; CSV Aggregation; Log Parsing	3	Parsing, malformed data, runtime, memory
Stateful processing	Production-oriented	Event Deduplication; LRU Cache	2	State handling, correctness, maintainability
Resilience & control	Production-oriented	Retry/Backoff; Rate Limiter	2	Failure handling, timing, robustness
Validation & batching	Production-oriented	Input Validation; Batch Processing	2	Input resilience, batching, scalability

3.3. Models and Sampling Protocol

Three GPT-5.6 configurations were evaluated: Sol, Terra, and Luna. Every configuration received the same task prompt, and five independent generations were collected for each model-task pair. This produced 75 programs per configuration and 225 generated programs in total. Each task also had one fixed reference implementation, giving 240 evaluated implementations. Generated code was preserved without manual repair. The run also retained model labels, prompts, prompt hashes, pricing assumptions, raw responses, and environment metadata. All 225 generation calls completed successfully, so generation failures were not mixed with behavioral results. The three configurations were intentionally selected from one model family and provider to reduce cross-provider and architecture differences while isolating configuration-level quality, strategy, and cost effects under a matched protocol. This controlled choice improves internal comparability but limits generalization to other model families. Table 3 summarizes these controls.

3.3.1. Prompt and Generation Protocol

Each task used a fixed prompt that specified the required function or program behavior, input/output expectations, task constraints, and, where relevant, the intended algorithmic strategy. The identical task prompt was supplied independently to Sol, Terra, and Luna; there was no conversational refinement, repair request, or feedback loop after generation. Five independent generation calls were made for every model-task pair. The returned source code was stored exactly as generated and frozen before nominal, robustness, stress, metamorphic, static-analysis, and performance evaluation. This protocol represents the LLM interaction used in the study while avoiding prompt adaptation that could confound comparisons.

Table 3: Study Configuration and Evaluation Protocol

Parameter	**Setting**	**Rationale**
LLM configurations	GPT-5.6 Sol, GPT-5.6 Terra, GPT-5.6 Luna	Cross-model comparison under matched reasoning effort
Workload	15 tasks: 6 algorithmic; 9 production-oriented	Mixed workload coverage
Sampling and corpus	5 generations per model-task pair; 225 LLM programs + 15 references = 240 implementations	Repeated sampling; raw outputs preserved; no manual repair
Nominal validation	800 functional checks + 368 applicable robustness checks across all implementations; all passed	Nominal behavior and hidden edge cases
Production stress	Task-specific moderate + severe profiles; combined production-stress rate and PRG	Expose readiness loss after nominal success
Metamorphic / strategy	Task-appropriate invariants + source/AST strategy classification	Behavioral consistency and implementation equivalence

Parameter	Setting	Rationale
Execution protocol	Same machine/harness; 2 warm-ups + 7 measured repetitions; fixed references	Comparable and reference-normalized evidence
Energy telemetry	Not included	No reliable direct hardware telemetry available

3.4. Evaluation Dimensions

Evaluation was carried out in six layers. Layer 1 checks syntax and nominal correctness. Layer 2 adds hidden edge cases. Layer 3 applies separate moderate and severe production-like stress profiles. Layer 4 measures runtime and peak memory against a fixed task reference. Layer 5 records cyclomatic complexity, maintainability, and Bandit findings. Layer 6 adds metamorphic checks where appropriate, implementation-strategy classification, and generation cost. For each generated program, production-stress success is calculated as the number of passed applicable moderate and severe stress checks divided by the total number of applicable stress checks for that program. Model-level stress rates summarize these program-level rates across the 75 generations for each configuration; task-level rates are summarized across the applicable generations for that task. Moderate and severe sets are task-specific and are not copies of nominal tests with larger input sizes.

The Production-Readiness Gap is defined directly from the two behavioral layers:

$$PRG = R_{nominal} - R_{stress} \quad (1)$$

Here, $R_{nominal}$ is nominal functional success and R_{stress} is the combined pass rate across the applicable moderate and severe stress checks. A larger PRG means that more behavior was lost when the evaluation moved from nominal tests to production-like stress.

Table 4: Representative Moderate and Severe Production-Like Stress Profiles

Task	Moderate stress profile	Severe stress profile	Evaluation purpose
CSV Aggregation	Empty CSV plus mixed valid, non-numeric, and blank-category rows	8,000 rows distributed across 20 categories	Malformed-data handling and volume resilience
JSON Normalization	Empty/mixed scalar or malformed records and duplicate tags	4,000 structured records with whitespace, duplicate tags, and boolean fields	Normalization correctness under malformed inputs and scale
Input Validation	None/empty objects, boolean age, oversized username, and age 131	Malformed email edge cases (double-@ and empty domain label)	Boundary and format strictness beyond nominal examples

Task	Moderate stress profile	Severe stress profile	Evaluation purpose
Event Deduplication	Empty/invalid records, unhashable identifiers, and duplicate IDs	5,000 events with 50 repeating identifiers	State/identifier handling and repeated-event scale

Table 4 gives representative stress cases for the four tasks with the largest observed readiness gaps. The candidate and the fixed reference are run against the same profile. The full harness keeps the exact cases for all 15 tasks; the table shows enough detail to explain how moderate and severe conditions differ without listing every test case in the paper.



Figure 1: Methodology block diagram for the production-readiness testbed, sampling, controls, and evidence layers

3.5. Reference Implementations and Execution Protocol

Each task has a fixed reference implementation. The references are not claimed to be universally optimal solutions; they provide stable task-level anchors. Generated and reference programs use the same machine and harness. Runtime uses two warm-up executions followed by seven measured repetitions, and peak memory is captured during execution. Runtime and memory are reported as ratios to the task reference, while maintainability is reported as a difference from the reference. Raw generations, configuration snapshots, prompt hashes, and measurement files were retained. Missing benchmark values remain missing rather than being filled in. CPU governor locking and container-level CPU pinning were not enforced; therefore, sub-millisecond timings should be interpreted as controlled-environment observations rather than hardware-isolated benchmarks. The greater-than-500x Quick Sort spread is used as evidence of implementation-strategy divergence, not as a claim of a precisely hardware-normalized speedup.

3.6. Statistical Analysis

The analysis uses both descriptive summaries and nonparametric statistical tests. Runtime, memory, maintainability, complexity, stress outcomes, PRG, and cost are summarized by model, task, and workload class. Kruskal-Wallis is used for cross-model comparisons when the measure

has variation. Pairwise comparisons use two-sided Mann-Whitney U tests with Holm correction, and Cliff's delta reports effect size. Bootstrap 95% confidence intervals are reported for production-stress success and PRG. No inferential test is interpreted when every observation is the same, as occurred for nominal success and the applicable metamorphic evaluations. When one Luna Quick Sort runtime and peak-memory measurement was unavailable, that observation was excluded only from the affected runtime and memory summaries and statistical comparisons; no value was imputed, and the program's remaining behavioral, structural, strategy, and cost evidence was retained.

4. EXPERIMENTAL EVALUATION

4.1. Experimental Setup

The final run contains 15 tasks, three model configurations, and five generations for every model-task pair. The result is 225 generated programs plus 15 fixed references. Across the 240 implementations, 800 functional checks and 368 applicable hidden-edge-case checks were executed; all passed. The generated programs account for 750 of the functional checks and 345 of the hidden-edge-case checks. After that stage, the frozen programs were evaluated with moderate and severe stress and 105 applicable metamorphic checks. Energy was not included because direct hardware telemetry was not collected. One Luna Quick Sort program did not produce a usable runtime or peak-memory measurement; its code and the remaining evidence were retained, and the missing values were not imputed.

4.2. Overall Production-Readiness Results

Nominal testing did not separate the models. All 225 generated programs were syntactically valid, and every nominal functional and applicable hidden-edge-case check passed. The stronger stress layer produced a different result. Production-stress success was 95.16% for Sol, 95.60% for Terra, and 93.38% for Luna, which corresponds to PRGs of 4.84%, 4.40%, and 6.62%. The confidence intervals overlap, and the cross-model difference is not statistically significant. All 105 applicable metamorphic evaluations passed. Table 5 summarizes these results.

Table 5: Production-Readiness Profile of Generated Software

Model	Nom. success	Prod. stress	Stress CI	PRG	PRG CI	Metamorphic	Gen. cost
GPT-5.6 Sol	100%	95.16%	92.75-97.29%	4.84%	2.62-7.27%	100%	$0.5109
GPT-5.6 Terra	100%	95.60%	93.30-97.57%	4.40%	2.33-6.79%	100%	$0.3190
GPT-5.6 Luna	100%	93.38%	90.49-95.83%	6.62%	3.98-9.37%	100%	$0.0422

The main result in Table 5 is the gap between nominal and stress behavior. All three configurations reached 100% nominal success, but none reached 100% production-stress success. Sol and Terra remained near 95%, while Luna was slightly above 93%. Because the confidence intervals overlap and the omnibus test is not significant, these values should not be read as a model ranking. PRG is used here to show hidden readiness loss after nominal success. The 105 applicable metamorphic checks remained saturated at 100%.

4.3. Workload-Class Analysis

The workload classes were compared to see whether results from canonical algorithms carry over to more operational tasks. Algorithmic tasks are controlled and useful for exposing implementation-level performance differences. Production-oriented tasks add parsing, state, failure handling, traffic control, validation, and batching. Table 6 shows that both classes reached the same nominal and hidden-edge-case ceiling, but their resource and structural profiles were different.

Table 6: Behavioral Quality Across Algorithmic and Production-Oriented Workloads

Model	Workload	Nominal	Edge-case	Runtime (ms)	Peak mem. (KB)	Cycl. comp.	Maintain.	Gen. cost
Sol	Algorithmic	100%	100%	212.09	414.11	4.67	70.93	$0.00411
Sol	Production	100%	100%	64.64	1965.48	6.77	59.69	$0.00861
Terra	Algorithmic	100%	100%	208.16	473.01	4.57	59.97	$0.00251
Terra	Production	100%	100%	52.99	1940.11	6.35	59.64	$0.00541
Luna	Algorithmic	100%	100%	194.73	355.67	5.87	60.38	$0.00051
Luna	Production	100%	100%	49.28	1966.15	6.39	60.44	$0.00060

Table 6 shows the workload-level pattern. Behavior and hidden-edge-case resilience are 100% in all six model-workload combinations. Runtime, peak memory, complexity, maintainability, and generation cost do not follow the same pattern. In particular, the production-oriented tasks use substantially more peak memory on average and are structurally more complex. The stress results also show where the readiness loss occurred. Pooled across models, CSV Aggregation had a 31.11% PRG, JSON Normalization 25.00%, Input Validation 14.29%, and Event Deduplication 8.89%. The other tasks had no aggregate gap under the implemented stress profiles. This task-level view is more useful than simply labeling one workload class as harder because it identifies the specific behaviors that failed after nominal correctness had already reached 100%.

4.4. Reference-Normalized Analysis

Fixed references provide a task-level baseline for the non-functional results. A ratio near 1.0 means the generated value is close to the reference; a value above 1.0 is larger, and a value below 1.0 is smaller. Maintainability is reported as a delta from the reference. Table 7 shows that most median runtime and memory ratios remain near the task baselines. Sol and Luna use lower median memory on the algorithmic tasks, while the production-oriented memory ratios are close to 1.0 for all three configurations. These values are descriptive because a different implementation strategy can trade one resource for another.

Table 7: Reference-to-Generated Software Performance Ratios

Model	Workload	Runtime ratio vs ref.	Memory ratio vs ref.	Maintainability Δ	Interpretation
Sol	Algorithmic	0.987x	0.681x	-1.95	Near-reference runtime; lower median memory
Sol	Production	1.069x	1.000x	-2.76	Small runtime overhead; reference-level memory
Terra	Algorithmic	1.000x	1.000x	-2.28	Near-reference runtime and memory
Terra	Production	0.987x	1.000x	-1.94	Slightly lower runtime; reference-level memory
Luna	Algorithmic	0.935x	0.726x	-3.16	Lower median runtime and memory
Luna	Production	0.954x	1.000x	-2.46	Slightly lower runtime; reference-level memory

4.5. Statistical Evidence

The omnibus tests do not support a cross-model difference for production-stress rate or PRG ($H = 1.398$, $p = 0.497$ for both). Runtime ratio, memory ratio, and maintainability delta are also not significant. Nominal success and metamorphic success have no cross-model variation, so no inferential comparison is reported for them. Bandit found no issues in the 225 generated outputs; one finding occurred in a fixed reference and is not attributed to a model. Generation cost is the clear exception, with $H = 153.771$ and $p = 4.07 \times 10^{-34}$. Table 8 separates the statistically supported result from the descriptive differences.

Table 8: Cross-Model Statistical Evidence for the Final Run

Metric	Kruskal-Wallis statistic	p-value	Interpretation
Nominal / edge-case success	No variation	-	100% across generated programs / applicable checks
Production-stress rate	1.398	0.4970	Not significant
Readiness gap	1.398	0.4970	Not significant
Runtime ratio vs reference	3.267	0.1953	Not significant

Metric	Kruskal-Wallis statistic	p-value	Interpretation
Memory ratio vs reference	1.823	0.4019	Not significant
Maintainability Δ vs reference	1.232	0.5402	Not significant
Metamorphic rate	No variation	-	100% across 105 applicable LLM evaluations
API generation cost	153.771	4.07×10^{-34}	Significant

Since cost is the only aggregate measure with a strong cross-model difference, Table 9 reports the pairwise cost comparisons. The Sol-Luna and Terra-Luna effects are very large, while the Sol-Terra effect is medium after Holm correction.

Table 9: Pairwise Model Differences and Effect Sizes

Comparison	Metric	Holm-adjusted p-value	Cliff's δ	Effect
Sol vs Luna	Generation cost	1.34×10^{-25}	0.999	Very large
Terra vs Luna	Generation cost	1.17×10^{-24}	0.976	Very large
Sol vs Terra	Generation cost	3.58×10^{-5}	0.391	Medium

The cost result does not mean that the least expensive configuration is always the best choice. In this workload, paying more did not produce a statistically supported increase in aggregate production-stress success. A deployment decision still depends on the properties that matter for the target workload. Higher cost is useful only when it produces a measurable advantage on an acceptance criterion that matters.

4.6. Quality-Cost Trade-Off

Table 10 places the quality and cost results side by side. Nominal success is 100% for every configuration, production-stress success stays between 93.38% and 95.60%, and PRG stays between 4.40% and 6.62%. Cost behaves very differently: the total cost for Sol is more than an order of magnitude above Luna.

Table 10: Quality-Cost Trade-Offs Across Evaluated LLMs

Model	Nominal / production stress	Readiness gap	Median runtime (ms)	Median peak memory (KB)	Total cost
GPT-5.6 Sol	100% / 95.16%	4.84%	18.20	571.36	$0.5109
GPT-5.6 Terra	100% / 95.60%	4.40%	67.88	681.13	$0.3190
GPT-5.6 Luna	100% / 93.38%	6.62%	52.54	597.00	$0.0422

Table 10 also includes the absolute runtime and memory medians. Sol has the lowest observed medians at 18.20 ms and 571.36 KB; Luna is between Sol and Terra on these two values. These are descriptive observations because the aggregate runtime and memory differences are not statistically significant. Luna's medians use 59 benchmark-applicable observations because one Quick Sort benchmark was unavailable; Sol and Terra each use 60. The pairwise cost effects remain the strongest cross-model result. The combined result does not identify a universal winner. Luna produced the least expensive corpus and still remained above 93% on production stress, but the stress difference is not significant and some workloads may value runtime, memory, or implementation strategy more heavily. Sol has lower observed runtime and memory medians, but those differences are not statistically supported at the aggregate level. The useful decision is therefore workload-specific rather than a single overall ranking.

4.7. Implementation-Strategy Divergence

Correct behavior can also hide a different implementation strategy. Quick Sort provides the clearest example. All five Sol generations delegated sorting to Python's built-in sorted function and passed the same nominal and stress checks; their runtimes were about 0.297–0.450 ms. Terra generation 1 implemented recursive Quick Sort explicitly and took 167.334 ms while still passing the same checks. Comparing the 0.297 ms Sol case with the 167.334 ms Terra case gives a spread greater than 500× even though the observed outputs were correct.

Table 11: Implementation-Strategy Deviations Observed in Generated Programs

Task / generation	**Requested behavior**	**Observed strategy**	**Behavioral outcome**	**Non-functional consequence**
Quick Sort, Sol Gen. 1	Ascending sort	Divergent: built-in sorted(values)	100% nominal and stress	0.324 ms
Quick Sort, Sol Gen. 2	Ascending sort	Divergent: built-in sorted(values)	100% nominal and stress	0.297 ms
Quick Sort, Terra Gen. 1	Ascending sort	Conformant: explicit recursive Quick Sort	100% nominal and stress	167.334 ms

Table 11 and Figure 2 show representative implementations from this case. The purpose is not to claim that one model is globally faster. The point is that behavioral equivalence does not guarantee implementation equivalence. A pass/fail test cannot show whether the model followed the requested algorithm, delegated the work to a library, or chose another strategy.

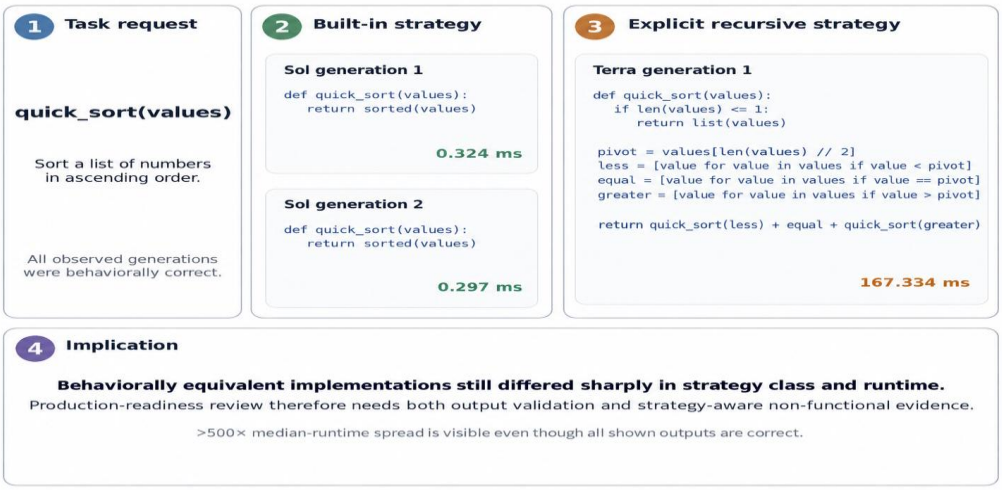


Figure 2: Quick Sort strategy divergence among behaviorally equivalent generations

The strategy classifier applied to 105 outputs: 62 were conformant, 28 alternate, 7 divergent, and 8 required review. The remaining 120 outputs were outside the strategy-specific scope of the classifier. The Quick Sort example shows why this evidence matters. A library-delegating implementation and an explicit recursive implementation can both pass the tests and still have very different runtime behavior. Strategy classification is therefore used with the behavioral and non-functional results, not as a standalone quality score.

5. DISCUSSION

The final run shows why nominal correctness alone is insufficient for production-oriented review. Every generated program passed the nominal and applicable hidden-edge-case checks, yet production stress exposed readiness gaps of 4.40%-6.62%. The workload-level failures were concentrated in CSV Aggregation, JSON Normalization, Input Validation, and Event Deduplication, while production-oriented tasks also showed different memory and complexity profiles. These results indicate that success on standard algorithms should not be treated as evidence that parsing, stateful processing, validation, and other operational code will behave similarly under stronger conditions.

The economic and implementation evidence adds a second distinction. Generation cost differed strongly, whereas aggregate production-stress, runtime, memory, and maintainability differences were not statistically significant. A higher generation cost is therefore useful only when it produces a measurable advantage on criteria that matter for the target workload. The Quick Sort case likewise shows that behaviorally correct and stress-resilient programs can use very different computational strategies, so review should consider output behavior together with implementation acceptability for performance, auditability, maintainability, and future change.

The framework also has a direct illustrative application to IoT and edge/cloud software, where generated code may parse telemetry, normalize heterogeneous payloads, validate device inputs, deduplicate events, enforce rate limits, or batch records before downstream processing. IoT architectures must operate across heterogeneous devices, event-driven communication, resource constraints, and quality attributes such as reliability and performance [24]. Applying the present layered evaluation to generated IoT components could therefore expose failures that remain hidden under nominal tests. This study did not execute a dedicated IoT workload, so this is an application of the evaluation method rather than an additional experimental result.

The study is intentionally controlled. It uses 15 Python tasks, three configurations from one model family, five generations per model-task pair, and 15 fixed references. Restricting the experiment to one model family reduces provider-level confounding but does not establish that the same patterns hold across other model families, languages, repositories, or deployment stacks. The moderate and severe profiles are task-specific synthetic stress tests: they expose concrete weaknesses beyond nominal tests but cannot reproduce the full concurrency, dependency failures, adversarial traffic, data drift, observability constraints, or operational interactions of a live production system. Accordingly, the Production-Readiness Gap is evidence of readiness loss under the implemented testbed, not proof of deployment safety. Metamorphic checks apply only where a meaningful invariant exists. Bandit findings are static indicators and cannot substitute for dynamic security testing, fuzzing, or malicious-payload evaluation. Direct hardware energy telemetry was not collected, and one Luna Quick Sort runtime and memory measurement was unavailable and left missing. Because model-level stress differences are not significant, the evidence supports the evaluation method and observed readiness gaps rather than a definitive model ranking.

The study artifacts were retained for reproducibility, including the 15 task specifications and prompts, 15 fixed references, all 225 generated source files, raw response metadata and prompt hashes, configuration snapshots, nominal and robustness measurements, stress and metamorphic logic, static-analysis outputs, and the analysis scripts used for the tables and figures. API credentials are excluded. These materials can be provided to editors or reviewers and can be deposited in a public archive with the final publication version.

6. CONCLUSIONS AND FUTURE WORK

This study evaluated 225 LLM-generated programs and 15 fixed references across 15 tasks after nominal correctness had reached a ceiling. All 800 functional checks and 368 applicable hidden-edge-case checks passed, but production-stress success fell to 95.16% for Sol, 95.60% for Terra, and 93.38% for Luna, corresponding to PRGs of 4.84%, 4.40%, and 6.62%. All 105 applicable metamorphic checks passed. The cross-model production-stress, runtime, memory, and maintainability differences were not statistically significant, while generation cost differed strongly. At the task level, readiness loss concentrated in four production-oriented tasks, and the Quick Sort case showed that two correct implementations could differ by more than 500x in runtime because they used different strategies. The central implication is that generated code should be reviewed as software using stress behavior, task references, resource evidence, structural quality, implementation strategy, and cost rather than nominal correctness alone. Future work should apply the same evaluation structure to additional model families, languages, larger repository- and service-level workloads, IoT/edge scenarios, and continuous-integration or deployment environments. Direct energy measurement using interfaces such as Intel RAPL where available, or external hardware power meters, would also allow generation and execution energy to be measured rather than inferred.

REFERENCES

[1] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, and Q. Wang, “Software testing with large language models: Survey,landscape, and vision,” IEEE Transactions on Software Engineering, vol. 50, no. 4, pp. 911–936, 2024. https://doi.org/10.1109/TSE.2024.3368208

[2] M. Schaefer, S. Nadi, A. Eghbali, and F. Tip, “An empirical evaluation of using large language models for automated unit test generation,” IEEE Transactions on Software Engineering, vol. 50, no. 1, pp. 85–105, 2024. https://doi.org/10.1109/TSE.2023.3334955

[3] F. Tip, J. Bell, and M. Schaefer, "LLMorpheus: Mutation testing using large language models," IEEE Transactions on Software Engineering, vol. 51, no. 6, pp. 1645–1665, 2025. https://doi.org/10.1109/TSE.2025.3562025

[4] A. Silva, S. Fang, and M. Monperrus, "RepairLLaMA: Efficient representations and fine-tuned adapters for program repair," IEEE Transactions on Software Engineering, vol. 51, no. 8, pp. 2366–2380, 2025. https://doi.org/10.1109/TSE.2025.3581062

[5] O. Ben Sghaier, M. Weyssow, and H. A. Sahraoui, "Harnessing large language models for curated code reviews," in Proc. 2025 IEEE/ACM 22nd Int. Conf. Mining Software Repositories (MSR), pp. 187–198, 2025. https://doi.org/10.1109/MSR66628.2025.00039

[6] T. Y. Zhuo, J. He, J. Sun, Z. Xing, D. Lo, J. Grundy, and X. Du, "Identifying and mitigating API misuse in large language models," IEEE Transactions on Software Engineering, vol. 52, no. 3, pp. 855–873, 2026. https://doi.org/10.1109/TSE.2026.3651566

[7] D. Nichols, P. Polasam, H. Menon, A. Marathe, T. Gamblin, and A. Bhatele, "Performance-aligned LLMs for generating fast HPC code," IEEE Transactions on Parallel and Distributed Systems, vol. 37, no. 6, pp. 1330–1341, 2026. https://doi.org/10.1109/TPDS.2026.3675550

[8] J. Zhao, Y. Sun, C. Huang, C. Liu, Y. Guan, Y. Zeng, and Y. Liu, "Towards secure code generation with LLMs: A study on Common Weakness Enumeration," IEEE Transactions on Software Engineering, vol. 51, no. 12, pp. 3507–3523, 2025. https://doi.org/10.1109/TSE.2025.3619281

[9] H. Nunes, E. Figueiredo, L. Rocha, S. Nadi, F. Ferreira, and G. Esteves, "Evaluating the effectiveness of LLMs in fixing maintainability issues in real-world projects," in Proc. 2025 IEEE Int. Conf. Software Analysis, Evolution and Reengineering (SANER), 2025. https://doi.org/10.1109/SANER64311.2025.00069

[10] J. Holl and J. Kufer, "Holistic evaluation of LLM-based code generation," in Proc. 2025 3rd Int. Conf. Foundation and Large Language Models (FLLM), pp. 234–241, 2025. https://doi.org/10.1109/FLLM67465.2025.11390916

[11] N. Nascimento, E. Guimaraes, S. S. Chintakunta, and S. A. Boominathan, "How effective are LLMs for data science coding? A controlled experiment," in Proc. 2025 IEEE/ACM 22nd Int. Conf. Mining Software Repositories (MSR), pp. 211–222, 2025. https://doi.org/10.1109/MSR66628.2025.00041

[12] A. E. Pakis and A. Kolukisa, "Code generation in LLM families: A quality comparison of reasoner and regular models," in Proc. 2025 10th Int. Conf. Computer Science and Engineering (UBMK), 2025. https://doi.org/10.1109/UBMK67458.2025.11207012

[13] S. Honarvar, M. Rei, and A. F. Donaldson, "The question neighbourhood approach for systematic evaluation of code-generating LLMs," IEEE Transactions on Software Engineering, vol. 51, no. 11, pp. 3138–3167, 2025. https://doi.org/10.1109/TSE.2025.3612251

[14] R. Ravi, D. Bradshaw, S. Ruberto, G. Jahangirova, and V. Terragni, "LLMLOOP: Improving LLM-generated code and tests through automated iterative feedback loops," in Proc. 2025 IEEE Int. Conf. Software Maintenance and Evolution (ICSME), 2025. https://doi.org/10.1109/ICSME64153.2025.00109

[15] A. Mhatre, N. Nader, P. Diehl, and D. Gupta, "Can LLMs find bugs in code? An evaluation from beginner errors to security vulnerabilities in Python and C++," in Proc. SoutheastCon 2026, 2026. https://doi.org/10.1109/SoutheastCon63549.2026.11476483

[16] I. Jaoua, O. Ben Sghaier, and H. A. Sahraoui, "Combining large language models with static analyzers for code review generation," in Proc. 2025 IEEE/ACM 22nd Int. Conf. Mining Software Repositories (MSR), pp. 174–186, 2025. https://doi.org/10.1109/MSR66628.2025.00038

[17] F. S. Adalsteinsson, B. B. Magnusson, M. Milicevic, A. N. Davidsson, and C.-H. Cheng, "Rethinking code review workflows with LLM assistance: An empirical study," in Proc. 2025 ACM/IEEE Int. Symp. Empirical Software Engineering and Measurement (ESEM), pp. 488–497, 2025.

[18] Y. Tang, Y. Liu, J. Lan, Z. Yan, and E. Gelenbe, "Security of LLM-based agents regarding attacks, defenses, and applications: A comprehensive survey," Information Fusion, vol. 127, Art. no. 103941, 2026. https://doi.org/10.1016/j.inffus.2025.103941

[19] D. Cotroneo, C. Improta, and P. Liguori, "Human-written vs. AI-generated code: A large-scale study of defects, vulnerabilities, and complexity," in Proc. 2025 IEEE 36th Int. Symp. Software Reliability Engineering (ISSRE), 2025.

[20] S. G. Patil, H. Mao, F. Yan, C. C.-J. Ji, V. Suresh, I. Stoica, and J. E. Gonzalez, "The Berkeley Function Calling Leaderboard (BFCL): From tool use to agentic evaluation of large language models," in Proc. 42nd Int. Conf. Machine Learning, vol. 267, pp. 48371–48392, 2025.

[21] P. S. Samal, S. K. Palus, and S. K. Padmam, “From benchmark scores to deployment readiness: A journal-scale evaluation framework for autonomous software development agents,” International Journal of Software Engineering & Applications, vol. 17, no. 2, pp. 1–21, 2026. https://doi.org/10.5121/ijsea.2026.17201

[22] V. V. Yadavali, “AI-driven performance testing framework for mobile applications,” International Journal of Software Engineering & Applications, vol. 15, no. 6, pp. 33–45, 2024. https://doi.org/10.5121/ijsea.2024.15603

[23] R. Khankhoje, “An intelligent API testing: Unleashing the power of AI,” International Journal of Software Engineering & Applications, vol. 15, no. 1, pp. 1–8, 2024. https://doi.org/10.5121/ijsea.2024.15101

[24] Y. Abuseta, “Towards a generic software architecture for IoT systems,” International Journal of Software Engineering & Applications, vol. 15, no. 6, pp. 1–16, 2024. https://doi.org/10.5121/ijsea.2024.15601